



CST1500

Computer Systems Architecture and Operating Systems

Coursework 2: Report

Student Names: Syed Mohammed Jafer Hussain, Amro Asy, Saifana Kuradiela Maryam

MISIS: M01087633, M01033626, M01088086

Module Lecturer: Ms. Madhumita Das

Submission Date: 02/11/2025

INDEX

1. INTRODUCTION	3
2. CPU SCHEDULING ALGORITHMS	4
2.1. First Come First Served (FCFS):	4
2.2. Shortest Job First (SJF):	4
2.3. Round Robin (RR)	4
3. FIRST COME FIRST SERVED (FCFS)	5
3.1. Approach:	5
3.2. Algorithm:	5
3.3. Code Screenshots:	6
3.4. Code Output Screenshots:	7
3.5. Output Explanation:	7
4. SHORTEST JOB FIRST (SJF)	8
4. 1. Approach:	8
4.2. Algorithm:	8
4.3. Code Screenshots:	8
4.4. Code Output:	11
4.5. Output Explanation:	12
5. ROUND ROBIN (RR)	13
5.1. Approach:	13
5.2. Algorithm:	13
5.3. Code Screenshots:	14
5.4. Code Output Screenshots:	15
5.5. Output Explanation:	16
6. CONCLUSION	17
7. BIBLIOGRAPHY	18
1. FCFS:	18
2. SJF:	18
3. RR	19

1. INTRODUCTION

This coursework project was done by the following: Saifana Kuradiela Maryam (M01088086) assigned to Round Robin (RR), Syed Mohammed Jafer Hussain (M01087633) assigned to Shortest Job First (SJF), and Amro Asy (M01033626) assigned to First Come First Serve (FCFS).

The work was distributed evenly between each task member when programming the algorithm, as well as documenting their results and findings into this report. Each approach to the task has been noted for each individual CPU Scheduling Algorithm below.

2. CPU SCHEDULING ALGORITHMS

2.1. First Come First Served (FCFS):

The First-Come, First-Served (FCFS) scheduling method determines the waiting time and turnaround time for a set of processes according to their burst durations.

2.2. Shortest Job First (SJF):

Shortest Job First (SJF) scheduling is a very efficient CPU Scheduling Algorithm in which all processes are sorted in ascending order by burst time and executed from shortest burst time to longest burst time. This minimizes wait time for each process.

2.3. Round Robin (RR)

Round Robin (RR) scheduling is pre-emptive CPU Scheduling Algorithm that utilises the concept of circular ready queues. It utilises fixed time slices (time quantum) given to the processes to execute and is implemented in the ready queue pre-emptively. This ensures a starvation-free algorithm.

3. FIRST COME FIRST SERVED (FCFS)

3.1. Approach:

1. User Input:

The program asks the user for the number of processes and each process's burst time.

2. Waiting Time Calculation:

Each process's waiting time is computed by adding up the burst times of all processes before it.

3. Turnaround Time Calculation:

The turnaround time for every process is the sum of its burst time and its waiting time.

4. Averages Computed:

The program calculates the average waiting time and average turnaround time to show overall scheduling performance.

5. Results Displayed:

A table shows each process's ID, burst time, waiting time, and turnaround time, along with the overall averages, demonstrating how FCFS scheduling behaves.

3.2. Algorithm:

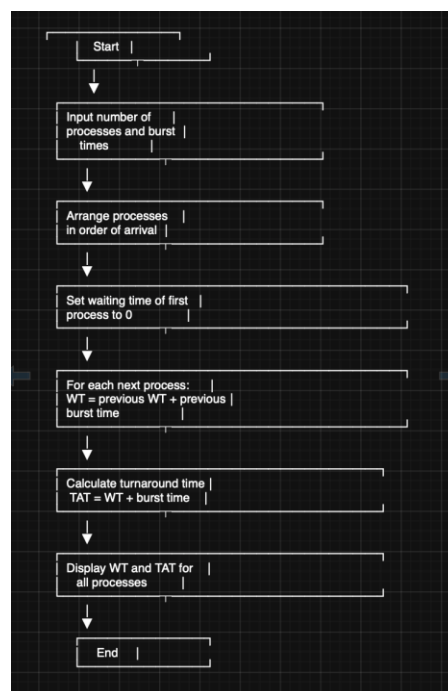


Fig 3.1: Flowchart of FCFS algorithm.

3.3. Code Screenshots:

```
fcfs code.py > ...
1  def calculate_fcfs(processes):
2      # Number of processes
3      n = len(processes)
4
5      # Initialize waiting time and turn-around time lists
6      waiting_time = [0] * n
7      turn_around_time = [0] * n
8
9      # Calculate waiting time
10     for i in range(1, n):
11         waiting_time[i] = processes[i - 1][1] + waiting_time[i - 1]
12
13     # Calculate turn-around time
14     for i in range(n):
15         turn_around_time[i] = processes[i][1] + waiting_time[i]
16
17     # Averages
18     avg_waiting_time = sum(waiting_time) / n
19     avg_turnaround_time = sum(turn_around_time) / n
20
21     return waiting_time, turn_around_time, avg_waiting_time, avg_turnaround_time
22
23
24     # -----
25     # Main program
26     # -----
27     n = int(input("Enter the number of processes: "))
28     processes = []
```

Fig 3.2: Code screenshot of main FCFS program. (1/2)

```

# Input burst times
for i in range(n):
    burst_time = int(input(f"Enter burst time for process {i+1}: "))
    processes.append((i + 1, burst_time))

# Calculate FCFS results
waiting_time, turn_around_time, avg_wait, avg_tat = calculate_fcfs(processes)

# Output results
print("\nProcess | Burst Time | Waiting Time | Turn Around Time")
print("-----")

for i in range(n):
    print(f"{processes[i][0]:<7} | {processes[i][1]:<10} | {waiting_time[i]:<12} | {turn_around_time[i]:<16}")

# Print averages
print(f"\nAverage Waiting Time: {avg_wait:.2f}")
print(f"Average Turn Around Time: {avg_tat:.2f}")
```

Fig 3.2: Code screenshot of main FCFS program. (2/2)

3.4. Code Output Screenshots:

```
Enter the number of processes: 3
Enter burst time for process 1: 4
Enter burst time for process 2: 6
Enter burst time for process 3: 7

Process | Burst Time | Waiting Time | Turn Around Time
-----|-----|-----|-----
1       | 4         | 0          | 4
2       | 6         | 4          | 10
3       | 7         | 10         | 17

Average Waiting Time: 4.67
Average Turn Around Time: 10.33
```

Fig 3.2: Output screenshot of main FCFS program.

3.5. Output Explanation:

The code is a straightforward implementation of the FCFS scheduling algorithm, which is one of the simplest types of CPU scheduling algorithms.

4. SHORTEST JOB FIRST (SJF)

4.1. Approach:

1. Use the threading module to simulate real threads
2. Create a user-defined subclass of Threads which will have burst time and waiting time for each thread
3. Get necessary data as user input, and use data to create subclass of Threads
4. Sort threads in ascending order of burst times and run them
5. Display output using any of the various modules that allow for tabulation

4.2. Algorithm:

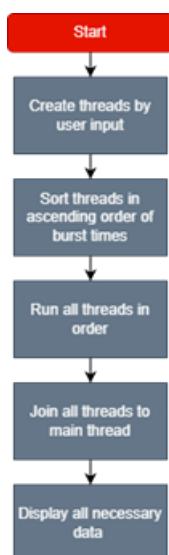


Fig 4.1: Main Algorithm Flowchart

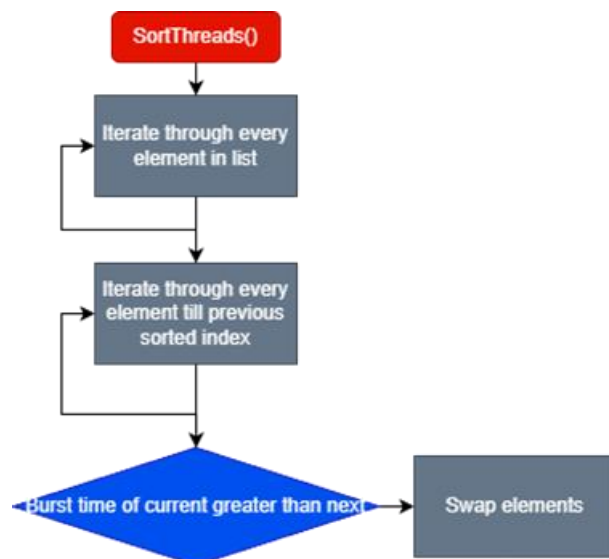


Fig 4.2: SortThreads() Algorithm Flowchart

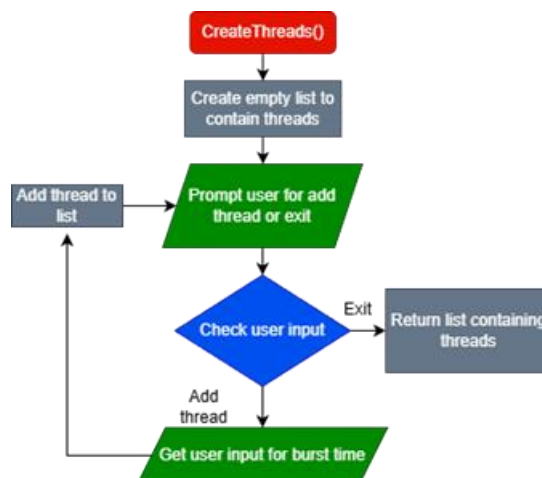


Fig 4.3: CreateThreads() Algorithm Flowchart

4.3. Code Screenshots:

```

1 | #CODE BY SYED MOHAMMED JAFER HUSSAIN M01087633
2 | from threading import Thread, Lock #Import necessary objects from modules
3 | from time import sleep
4 | from tabulate import tabulate
5 |
6 |
7 | class CWThreads (Thread): #Creating class CWThreads to contain thread burst time and thread number
8 |     def __init__ (self, burstTime: int, tNo: int):
9 |         """
10 |             Initialise CWThreads Object with burst time and thread number as parameter
11 |         """
12 |         Thread.__init__(self)
13 |         self.burstTime: int = burstTime
14 |         self.tNo: int = tNo
15 |         self.waitTime: int = 0
16 |         self.taT: int = 0 #Turnaround Time
17 |
18 |
19 |     def run(self) -> None: #Overwriting existing Thread.run() function
20 |         """
21 |             Using Lock().acquire() and .release() to allow for synchronisation between threads
22 |         """
23 |         tLock.acquire()
24 |         ExecThread(self)
25 |         tLock.release()
26 |

```

Fig 4.4: Code Screenshot Pt.1

```

26 |
27 |
28 | #CODE BY SYED MOHAMMED JAFER HUSSAIN M01087633
29 | def ExecThread(thread: CWThreads) -> None:
30 |     """
31 |         Prints thread started/ended and simulates thread running using time.sleep()
32 |         Returns: None
33 |     """
34 |     print(f"Thread {thread.tNo} started (BurstTime: {thread.burstTime})")
35 |     sleep(thread.burstTime)
36 |     print(f"Thread {thread.tNo} ended (BurstTime: {thread.burstTime})")
37 |
38 |
39 | def SortThreads(lst: list[CWThreads]) -> None:
40 |     """
41 |         Uses bubble sort algorithm to sort the threads in ascending order of burst time
42 |         Returns: None
43 |     """
44 |     lenLst: int = len(lst)
45 |
46 |     for i in range(lenLst):
47 |         swapped: bool = False
48 |         for j in range(0, lenLst - i - 1):
49 |             if lst[j].burstTime > lst[j + 1].burstTime:
50 |                 lst[j], lst[j+1] = lst[j+1], lst[j] #Swap if not in correct order
51 |                 swapped = True
52 |
53 |         if (not swapped):
54 |             break

```

Fig 4.5: Code Screenshot Pt.2

```

54         break
55 #CODE BY SYED MOHAMMED JAFER HUSSAIN M01087633
56
57
58 def CreateThreads() -> list[CWThreads]:
59     """
60     Using user input to get burst times and create threads
61     Returns: List containing all threads created
62     """
63     threads: list[CWThreads] = []
64     tNo: int = 1 #Thread Number
65     while True:
66         userCh: str = input("1) Add a new thread\n2) Exit Loop\n")
67         if userCh != "1": #Loop terminates when user gives input
68             break
69
70         burstTime: int = int(input("Enter burst time(secs): "))
71         threads.append(CWThreads(burstTime, tNo))
72
73         tNo += 1
74
75     return threads
76
77
78 #CODE BY SYED MOHAMMED JAFER HUSSAIN M01087633
79 def RunAndJoinThreads() -> None:
80     """
81     Runs all threads in for loop using .start() and then joins in another for loop using .join()
82     """

```

Fig 4.6: Code Screenshot Pt.3

```

82     """
83     for thread in threads:
84         thread.start()
85
86     for thread in threads:
87         thread.join()
88
89
90 ~def GetWaitingAndTaT():
91     """
92     Iterates through threads: list[CWThreads] and calculates waiting and turnaround time for each thread
93     """
94     waitTime = taT = totalWaitTime = totalTaT = 0 #Int values
95     for thread in threads:
96         taT += thread.burstTime #Get wait and turnaround time
97         thread.taT, thread.waitTime = taT, waitTime
98
99         totalWaitTime += waitTime #Increment to totalWaitTime
100        totalTaT += taT #Increment to total Turnaround Time
101
102        waitTime += thread.burstTime
103
104    global avgWaitTime
105    global avgTaT #Float values
106    avgWaitTime, avgTaT = totalWaitTime / len(threads), totalTaT / len(threads) #Get average of wait and turnaround time
107 #CODE BY SYED MOHAMMED JAFER HUSSAIN M01087633
108
109
110 ~def DisplayThreads() -> None:

```

Fig 4.7: Code Screenshot Pt.4

```

107 #CODE BY SYED MOHAMMED JAFER HUSSAIN M01087633
108
109
110 def DisplayThreads() -> None:
111     """
112     Creates a 2d matrix of form list[list] containing all details of every thread and prints a table using tabulate.tabulate()
113     """
114     tDetailsMatrix: list[list] = []
115     processOrder: str = ""
116     for thread in threads:
117         tDetailsMatrix.append([thread.tNo, thread.burstTime, thread.waitTime, thread.taT])
118         processOrder += f"Thread {thread.tNo} -> "
119
120     print( tabulate( tDetailsMatrix, headers = ["Process Number", "Burst Time", "Waiting Time", "Turnaround Time"], tablefmt = "psql" ) )
121     print( tabulate( [[avgWaitTime, avgTaT]], headers = ["Average Wait Time", "Average Turnaround Time"], tablefmt = "psql" ) )
122     print(processOrder[:-3])
123
124
125 if __name__ == "__main__": #In __main__
126     tLock: Lock = Lock() #Create lock for synchronisation of threads
127     threads: list[cwThreads] = CreateThreads() #Create all variables and store in global variable threads
128     SortThreads(threads)
129     RunAndJoinThreads()
130     GetWaitingAndTaT()
131     DisplayThreads() #Display final output after all calculations and simulations
132
133 #CODE BY SYED MOHAMMED JAFER HUSSAIN M01087633

```

Fig 4.8: Code Screenshot Pt.5

4.4. Code Output:

```

PS D:\Uni Work\Year 1\Sem 1\Comp Architecture\CST1500-CW2-> d:; cd 'd:\Uni Work\Year 1\
'---' 'D:\Uni Work\Year 1\Sem 1\Comp Architecture\CST1500-CW2-\Code\SJF.py'
1) Add a new thread
2) Exit Loop
1
Enter burst time(secs): 4
1) Add a new thread
2) Exit Loop
1
Enter burst time(secs): 6
1) Add a new thread
2) Exit Loop
1
Enter burst time(secs): 2
1) Add a new thread
2) Exit Loop
1
Enter burst time(secs): 2
1) Add a new thread
2) Exit Loop
1
Enter burst time(secs): 1
1) Add a new thread
2) Exit Loop
2
Thread 5 started (BurstTime: 1)
Thread 5 ended (BurstTime: 1)
Thread 3 started (BurstTime: 2)
Thread 3 ended (BurstTime: 2)
Thread 4 started (BurstTime: 2)
Thread 4 ended (BurstTime: 2)
Thread 1 started (BurstTime: 4)
Thread 1 ended (BurstTime: 4)
Thread 2 started (BurstTime: 6)
Thread 2 ended (BurstTime: 6)

```

Process Number	Burst Time	Waiting Time	Turnaround Time
5	1	0	1
3	2	1	3
4	2	3	5
1	4	5	9
2	6	9	15

Average Wait Time	Average Turnaround Time
3.6	6.6

```

Thread 5 -> Thread 3 -> Thread 4 -> Thread 1 -> Thread 2
PS D:\Uni Work\Year 1\Sem 1\Comp Architecture\CST1500-CW2->

```

Fig 4.9: Code Output Screenshot

4.5. Output Explanation:

The output shown above for SJF Scheduling displays how the program asks for clear input from the user about **burst time** and **exiting program** and displays a **neat table** with all relevant information.

The user is prompted for creating a new thread or to exit the loop, then after all loops are made, the program uses tabulate to create neat tables similar in format to SQL Tables having columns with heading **Process Number, Burst Time, Waiting Time, and Turnaround Time**. Then there's another table created to display the Average Wait Time and Average Turnaround Time. The program also outputs the process execution order at the end using arrows to indicate the order.

5. ROUND ROBIN (RR)

5.1. Approach:

My approach towards the RR algorithm was broken down into simple steps:

Creating the Round Robin Function (*round_robin_scheduling()*):

1. Taking in the arguments of the number of programs, arrival times, and burst times.
2. Utilising counters, lists and their index, to simulate a queue and track the programs with their corresponding *arrival* and *burst times*.
3. Sorting time quantum remainders into the queue from each program accordingly.
4. Employing the concept of flags to mark whether a program has been completed or unfinished according to the time quantum and remainders.

For Final Output and Main Code:

5. Taking in user inputs of the number of programs, arrival times, and burst times.
6. Calling the function and displaying the output in the format of a table for a clean, polished look.

5.2. Algorithm:

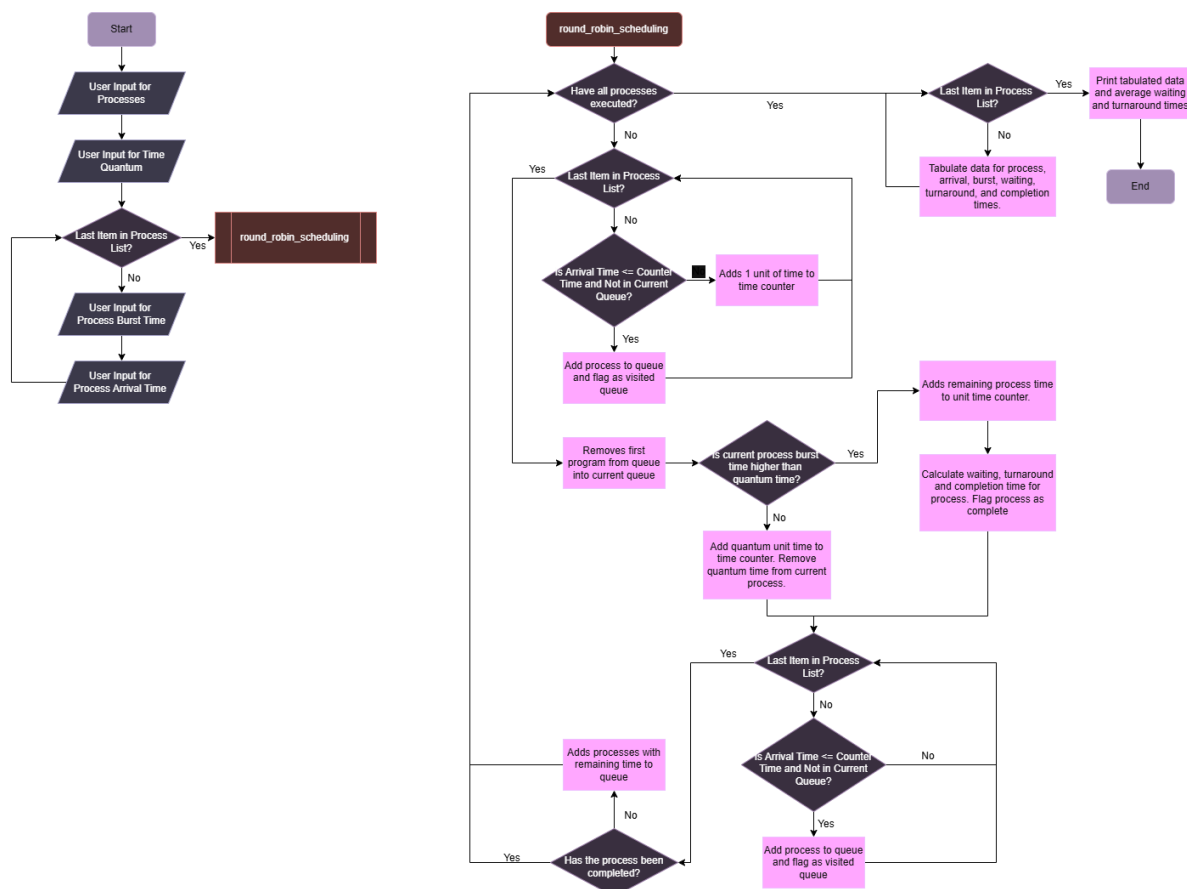


Figure 5.1: Main Program and Algorithm *round_robin_scheduling()* Function Flowcharts created in draw.io

5.3. Code Screenshots:

```
Round-Robin-CST1500-CW2.py > round_robin_scheduling
1 # CST1500 - CW2 - Round Robin Algorithm - Saifana Kuradiela Maryam - M01088086
2
3 from tabulate import tabulate # for table formatting, must be installed
4 # Round Robin Algorithm Function
5 def round_robin_scheduling(processes, arrival_times, burst_times, quantum):
6     # Variables to be used
7     n = len(processes)
8     # List slicing and indexing
9     remaining_bt = burst_times[:]
10    waiting_time = [0] * n
11    turnaround_time = [0] * n
12    completion_time = [0] * n
13    # Flags to check process statuses
14    visited = [False] * n # Checks if process is in queue
15    complete = [False] * n # Checks if process is executed fully
16    queue = [] # Empty queue list
17    time = 0 # Starting time of algorithm
18    # List to display Gantt-Chart process order
19    execution_order = []
20
21    while True:
22        # Adds new processes
23        for i in range(n):
24            if arrival_times[i] <= time and not visited[i]:
25                queue.append(i)
26                visited[i] = True
27        # Case where arrival nothing happens, or still waiting
28        if not queue:
29            time += 1
30            continue
31        # Removes first index of queue and stores in current
32        current = queue.pop(0)
33        # Adds the current process that is running to Gantt Chart
34        execution_order.append(processes[current])
```

Figure 5.2: Main Program Screenshot (1/3)

```
Round-Robin-CST1500-CW2.py > round_robin_scheduling
5 def round_robin_scheduling(processes, arrival_times, burst_times, quantum):
35     # Time quantum calculations and conditions
36     if remaining_bt[current] > quantum:
37         time += quantum
38         remaining_bt[current] -= quantum
39     else: # Calculating the TA and waiting times respective to TQ
40         time += remaining_bt[current]
41         waiting_time[current] = time - arrival_times[current] - burst_times[current]
42         turnaround_time[current] = time - arrival_times[current]
43         completion_time[current] = time
44         remaining_bt[current] = 0
45         complete[current] = True
46
47     # Add newly arrived processes during execution
48     for i in range(n):
49         if arrival_times[i] <= time and not visited[i]:
50             queue.append(i)
51             visited[i] = True
52
53     if not complete[current]: # Program that has not yet been executed
54         queue.append(current)
55
56     if all(complete): # ALL programs have been executed
57         break
58
59     # Tabulating data
60     table_data = []
61     for i in range(n):
62         table_data.append([
63             processes[i],
64             arrival_times[i],
65             burst_times[i],
66             waiting_time[i],
67             turnaround_time[i],
68             completion_time[i]
69         ])
70     headers = ["Process", "Arrival", "Burst", "Waiting", "Turnaround", "Completion"]
71     print("\n" + tabulate(table_data, headers=headers, tablefmt="grid"))
```

Figure 5.3: Main Program Screenshot (2/3)

```

Round-Robin-CST1500-CW2.py > round_robin_scheduling
5 def round_robin_scheduling(processes, arrival_times, burst_times, quantum):
72     avg_wait = sum(waiting_time) / n
73     avg_turn = sum(turnaround_time) / n
74     print(f"\nAverage Waiting Time: {avg_wait:.2f}")
75     print(f"\nAverage Turnaround Time: {avg_turn:.2f}")
76     # Printing the Execution Order
77     print("\nExecution Order:")
78     print(" -> ".join(execution_order))
79
80 # Variable Lists for RR function
81 listprocess = []
82 listburst = []
83 listarrival = []
84
85 # User input of processes, burst times and time quantum
86 quantum = int(input("Enter Time Quantum: "))
87 process = int(input("Enter Number of Processes: "))
88 for i in range(process):
89     burst = int(input(f"Enter Burst Time (P{i+1}): "))
90     arrival = int(input(f"Enter Arrival Time (P{i+1}): "))
91     listprocess.append("P" + str(i+1))
92     listburst.append(burst)
93     listarrival.append(arrival)
94
95 # Calling RR function
96 round_robin_scheduling(listprocess, listarrival, listburst, quantum)
97

```

Figure 5.4: Main Program Screenshot (3/3)

5.4. Code Output Screenshots:

```

Enter Time Quantum: 2
Enter Number of Processes: 4
Enter Burst Time (P1): 5
Enter Arrival Time (P1): 0
Enter Burst Time (P2): 3
Enter Arrival Time (P2): 5
Enter Burst Time (P3): 7
Enter Arrival Time (P3): 3
Enter Burst Time (P4): 2
Enter Arrival Time (P4): 6

```

Process	Arrival	Burst	Waiting	Turnaround	Completion
P1	0	5	2	7	7
P2	5	3	6	9	14
P3	3	7	7	14	17
P4	6	2	3	5	11

```

Average Waiting Time: 4.50
Average Turnaround Time: 8.75

Execution Order:
P1 -> P1 -> P3 -> P1 -> P2 -> P4 -> P3 -> P2 -> P3 -> P3

```

Figure 5.5: Main Program Output Screenshot

5.5. Output Explanation:

The output above displays the results of the Round Robin algorithm program. The program will first ask the user for a time quantum, the number of processes, the burst time and arrival time for each of the processes, then the average waiting and turnaround times.

The program then displays the calculated process number, arrival time, burst time, waiting time, turnaround time, as well as the completion time in a neat table created by the tabulate module. It also prints the execution order of the processes to display the order, akin to the Gantt chart.

6. CONCLUSION

This project completed by us three has been very informative and has really been a learning experience. I believe we learnt a lot with regards to team management, threading, CPU Scheduling Algorithms, and the threading module in Python.

Since this was a relatively simple project, we ran into very few issues. The concept of threading was properly taught to us and this helped in grasping the concept and implementing it in Python. Since we simulated threading using mainly lists, a very simple data type. the implementation went smoothly as well. The work to be done was clearly specified in MyLearning and that further sped up the process.

Some issues we faced would be figuring out the right approach for implementation and creating clear and concise flowcharts for the algorithms.

In conclusion, this group project has been completed successfully thanks to participation from all three team members as well as the guidance from our module tutor Ms. Madhumita Das.

7. BIBLIOGRAPHY

1. FCFS:

1. GeeksforGeeks. "Program for FCFS CPU Scheduling | Set 1." *GeeksforGeeks*, 22 Feb. 2017, www.geeksforgeeks.org/operating-systems/program-for-fcfs-cpu-scheduling-set-1/.
2. YashaswiniPrashanth. "Os-Programs-c." *Scribd*, 2025, www.scribd.com/document/700890656/os-programs-c. Accessed 12 Dec. 2025.
3. Peter, Anthony. "CPU Scheduling: Arrival, Burst, Completion, Turnaround, Waiting, and Response Time | Baeldung on Computer Science." *Www.baeldung.com*, 10 Jan. 2023, www.baeldung.com/cs/cpu-scheduling.

2. SJF:

1. Astanin, Sergey. "Tabulate: Pretty-Print Tabular Data." *PyPI*, 6 Oct. 2022, pypi.org/project/tabulate/. Accessed 9 Nov. 2025.
2. GeeksforGeeks. "Bubble Sort Algorithm." *GeeksforGeeks*, 2 Feb. 2014, www.geeksforgeeks.org/dsa/bubble-sort-algorithm/. Accessed 9 Nov. 2025.
3. "Difference between Turnaround Time (TAT) and Waiting Time (WT) in CPU Scheduling." *GeeksforGeeks*, 28 Mar. 2020, www.geeksforgeeks.org/operating-systems/difference-between-turn-around-time-tat-and-waiting-time-wt-in-cpu-scheduling/. Accessed 7 Nov. 2025.
4. Middlesex University. "2025-26 CST1500 Computer Systems Architecture and Operating Systems | MDX." *Mrooms.net*, 2025, mdx.mrooms.net/mod/resource/view.php?id=3352089. Accessed 7 Nov. 2025.

5. NeuralNine. “Python Threading Explained in 8 Minutes.” *Www.youtube.com*, 10 Jan. 2023, www.youtube.com/watch?v=A_Z1lgZLSNc. Accessed 7 Nov. 2025.
6. Python Software Foundation. “Threading — Thread-Based Parallelism — Python 3.9.0 Documentation.” *Docs.python.org*, 2025, docs.python.org/3/library/threading.html. Accessed 7 Nov. 2025.
7. Singh, Anshuman. “What Is Turnaround Time in CPU Scheduling?” *Shiksha.com*, 5 Feb. 2025, www.shiksha.com/online-courses/articles/turnaround-time-in-cpu-scheduling/. Accessed 9 Nov. 2025.

3. RR

1. Gupta, M. (2025). “Round Robin Scheduling in Operating System” *GeeksforGeeks.org*, *GeeksforGeeks*, <https://www.geeksforgeeks.org/operating-systems/round-robin-scheduling-in-operating-system/> Accessed 10 Nov. 2025.
2. JGraph Ltd and draw.io AG, (2017) *draw.io* (Version 29.0.3) [Diagram application], <https://www.drawio.com> Accessed 13 Nov. 2025.
3. Ouko, A. (2024). “Python Tabulate: A Full Guide” *DataCamp.com*, *DataCamp*, <https://www.datacamp.com/tutorial/python-tabulate> Accessed 17 Nov. 2025.
4. Saha, R. (2020). “Python List Slicing” *GeeksforGeeks.org*, *GeeksforGeeks*, <https://www.geeksforgeeks.org/python/python-list-slicing/> Accessed 10 Nov. 2025.